

COMPLETE ROADMAP & COURSEWORK

Zig

C's replacement — no hidden control flow, no hidden allocations

Why Zig exists, the allocator model, comptime, error unions, safety, and building real programs

7 Phases · Roadmap + Full Coursework with Code · Complete Reference

7 PHASES	40+ CONCEPTS	18 PROJECTS	~6mo TIMELINE	0 HIDDEN ALLOCATIONS
-------------	-----------------	----------------	------------------	----------------------------

PHASE 0 Why Zig Exists*The frustrations with C that led Andrew Kelley to build Zig*

0.1 The Problem with C

C is 50 years old. It runs the kernel, embedded devices, databases, and game engines. It is irreplaceable. But it has accumulated a set of footguns that cause bugs decade after decade — bugs that Zig was specifically designed to eliminate.

C Problem	Zig's Solution
Hidden allocations: malloc can be anywhere	All allocations require an explicit Allocator — no allocation is ever hidden
undefined behaviour is legal and silent	Zig has no undefined behaviour — illegal operations either panic or are compile errors
setjmp/longjmp for error handling causes impossible control flow	Errors are values in an error union — all error paths are visible and explicit
Header files, preprocessor macros, and the build system are separate worlds	Single language: comptime replaces macros; build.zig replaces Makefiles
Cross-compilation requires complex toolchain setup	zig cc is a drop-in C compiler that cross-compile to any target with one flag
No toolchain integration — need make, cmake, autoconf separately	zig build is a full build system written in Zig itself

Zig's Core Philosophy

No hidden control flow. No hidden allocations. No macros. No preprocessor. If it is not in the source code, it does not happen. Zig makes the implicit explicit — and that explicitness is where its power comes from.

Where Zig Fits

Use Case	Why Zig?
Systems programming / OS kernels	Direct hardware access, no runtime, no GC, full control over memory layout
Embedded / microcontrollers	No_stdlib target, tiny binary sizes, comptime for zero-cost abstractions
Game engines / real-time systems	Predictable performance — explicit allocators, no GC pauses, no hidden work
Replacing C in existing codebases	zig cc compiles C, Zig can call C directly, and you can rewrite incrementally

WebAssembly	Produces tiny .wasm files with no runtime overhead
High-performance tools and CLIs	Compiles to a single static binary with no dependencies

0.2 The Roadmap at a Glance

Phase 1 — Zig Fundamentals Variables, types, functions, control flow, and the Zig way of thinking.	Phase 2 — Memory Management Allocators, slices, pointers, defer, errdefer, and the ownership model.
Phase 3 — Error Handling Error sets, error unions, try, catch, errdefer — errors as values, properly.	Phase 4 — Comptime Compile-time execution: the most powerful feature in Zig — replaces macros and generics.
Phase 5 — Structs, Enums & Unions Tagged unions, methods, packed structs, extern structs, anonymous structs.	Phase 6 — Standard Library std.mem, std.fs, std.io, std.http, std.testing, std.ArrayList, std.HashMap.
Phase 7 — Build System & C Interop build.zig, cross-compilation, calling C from Zig, and the ecosystem.	

PHASE 1 Zig Fundamentals*Variables, types, functions, and Zig's explicit style*

1.1 Variables, Constants, and Types

Zig is extremely explicit. Every variable must have a known type at compile time, undefined values must be explicit, and there are no implicit conversions.

```
const std = @import("std");

pub fn main() void {
    // const - immutable, must be initialised
    const x: i32 = 42;
    const greeting = "hello"; // type inferred as *const [5:0]u8

    // var - mutable, must be initialised
    var count: u32 = 0;
    count += 1;

    // undefined - explicitly uninitialized (do NOT read before writing)
    var buffer: [1024]u8 = undefined;
    // Reading buffer before writing is safety-checked undefined behavior
    // (panics in safe mode, undefined in ReleaseFast)

    // Type inference with _ annotation
    const big: u64 = 1_000_000_000; // underscores allowed in numbers
    const pi = 3.14159;           // inferred as comptime_float

    // Explicit casts - no implicit conversions EVER
    const a: i32 = 5;
    const b: i64 = @intCast(a);    // explicit widening
    const c: i32 = @truncate(b);   // explicit truncation (may lose data)
    const f: f32 = @floatFromInt(a); // int to float
    const i: i32 = @intFromFloat(f); // float to int (truncates)

    _ = x; _ = greeting; _ = buffer; _ = big; _ = pi;
    _ = b; _ = c; _ = f; _ = i; _ = count;
}
```

Zig's Integer Types

Type	Description	Example values
i8 i16 i32 i64 i128	Signed integers	i32: -2,147,483,648 to 2,147,483,647
u8 u16 u32 u64 u128	Unsigned integers	u8: 0 to 255 (byte), u32: 0 to 4 billion

isize usize	Pointer-sized integers	Use usize for array lengths, pointer arithmetic
i3, u7, u1...	Arbitrary bit-width integers	Exact-width types for protocols, bit-packing
f16 f32 f64 f80 f128	IEEE 754 floats	f64 is the default for most calculations
bool	true or false	Stored as 1 byte but usable as 1-bit in packed structs
void	No value	Used as return type for functions with no return
noreturn	Function never returns	Used for functions that panic, loop forever, or exit

1.2 Functions

```

const std = @import("std");

// Basic function - return type always required
fn add(a: i32, b: i32) i32 {
    return a + b;
}

// Multiple return values via structs (Zig has no tuple return like Go)
const MinMax = struct { min: i32, max: i32 };
fn minMax(arr: []const i32) MinMax {
    var result = MinMax{ .min = arr[0], .max = arr[0] };
    for (arr) |v| {
        if (v < result.min) result.min = v;
        if (v > result.max) result.max = v;
    }
    return result;
}

// Functions are first-class values
fn apply(f: fn (i32) i32, x: i32) i32 {
    return f(x);
}
fn double(x: i32) i32 { return x * 2; }
const result = apply(double, 5); // 10

// Inline functions - always inlined at call site
inline fn max(a: anytype, b: anytype) @TypeOf(a) {
    return if (a > b) a else b;
}

// pub - export from module
pub fn publicFunction() void { }
fn privateFunction() void { } // only accessible within this file

// callconv - specify calling convention (for C interop, syscalls)

```

```
fn cCompatible(x: i32) callconv(.C) i32 { return x; }
```

1.3 Control Flow

```
// if – can be an expression
const val = if (x > 0) "positive" else "non-positive";

// if with optional unwrapping
if (maybeNull) |value| {
    // value is the unwrapped non-null value
    std.debug.print("{d}\n", .{value});
} else {
    std.debug.print("was null\n", .{});
}

// while loop
var i: usize = 0;
while (i < 10) : (i += 1) { // : (post_expr) runs after each iteration
    if (i == 5) continue;
    if (i == 8) break;
    std.debug.print("{d} ", .{i});
}

// for loop – iterates over arrays, slices, and ranges
const items = [_]i32{10, 20, 30, 40};
for (items) |item| { std.debug.print("{d}\n", .{item}); }
for (items, 0..) |item, index| { // with index
    std.debug.print("{d}: {d}\n", .{index, item});
}

// for over a range (Zig 0.12+)
for (0..10) |i| { std.debug.print("{d} ", .{i}); }

// switch – must be exhaustive
const colour = Colour.red;
switch (colour) {
    .red => std.debug.print("Red\n", .{}),
    .green => std.debug.print("Green\n", .{}),
    .blue => std.debug.print("Blue\n", .{}),
    // No default needed – Zig knows all enum cases are covered
}

// switch as expression
const hex = switch (colour) {
    .red => 0xFF0000,
    .green => 0x00FF00,
    .blue => 0x0000FF,
};
```

Phase 1 Projects



Number Theory Toolkit

Functions for: isPrime, primeFactors, gcd, lcm, fibonacci(n). No allocations. Uses loops, control flow, and functions only. Benchmark against a naive C implementation.

Concepts: functions · control flow · integer math · comptime_int · inline

PHASE 2 Memory Management*Allocators, pointers, slices, defer, and no hidden allocations*

2.1 The Allocator Model

Zig's most distinctive feature: there is no global allocator. Every function that needs to allocate heap memory must receive an Allocator as a parameter. This makes allocations visible, testable, and controllable.

```
const std = @import("std");
const Allocator = std.mem.Allocator;

// Functions that allocate take an allocator parameter
fn createNumbers(allocator: Allocator, count: usize) ![]i32 {
    const nums = try allocator.alloc(i32, count); // returns ![]i32
    for (nums, 0..) |*n, i| { n.* = @intCast(i * i); }
    return nums;
}

pub fn main() !void {
    // Choose allocator at call site – not baked into the function
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit(); // detects memory leaks on exit
    const allocator = gpa.allocator();

    const nums = try createNumbers(allocator, 10);
    defer allocator.free(nums); // always free – defer guarantees this

    for (nums) |n| { std.debug.print("{d} ", .{n}); }
}

// Testing with a fixed-size allocator (no heap, detects overflows)
test "createNumbers" {
    var buf: [1024]u8 = undefined;
    var fba = std.heap.FixedBufferAllocator.init(&buf);
    const nums = try createNumbers(fba.allocator(), 5);
    defer fba.allocator().free(nums);
    try std.testing.expectEqual(@as(i32, 0), nums[0]);
    try std.testing.expectEqual(@as(i32, 1), nums[1]);
}
```

Standard Allocators

Allocator	When to Use	Notes
GeneralPurposeAllocator	Development and production	Detects leaks and use-after-free in debug builds

ArenaAllocator	Batch allocations with one free	Allocate many things, free all at once — great for request handling
FixedBufferAllocator	Known maximum size, no heap	Stack-based, zero syscalls, perfect for embedded or tests
std.heap.page_allocator	Raw OS pages, rare	Large allocations, avoid for general use
std.testing allocator	In tests only	Detects leaks, checks for double-frees
c_allocator	Interfacing with C code	Wraps malloc/free — use only at C boundary

2.2 Pointers and Slices

```
// Single-item pointer: *T
var x: i32 = 42;
const ptr: *i32 = &x;
ptr.* = 100; // dereference with .*
std.debug.print("{d}\n", .{x}); // 100

// Many-item pointer: [*]T (no length info - unsafe, C-style)
const arr = [_i32{1, 2, 3, 4, 5};
const many_ptr: [*]const i32 = &arr;
const val = many_ptr[2]; // 3 - no bounds check!

// Slice: []T - pointer + length (preferred over many-item pointer)
// Always use slices - they carry bounds information
const slice: []const i32 = &arr; // converts array to slice
const part: []const i32 = arr[1..4]; // [2, 3, 4]
std.debug.print("len={d}\n", .{slice.len});

// String is a slice: []const u8 or []u8
const hello: []const u8 = "hello";
const world: []const u8 = "world";

// Null-terminated pointer: [*:0]const u8 (C string)
const c_str: [*:0]const u8 = "hello";

// Optional pointer: ?*T (null or a valid pointer)
var maybe: ?*i32 = null;
maybe = &x;
if (maybe) |p| { std.debug.print("{d}\n", .{p.*}); }

// Pointer alignment and size
const bytes align(16) = [_u8{0} ** 64; // 16-byte aligned
std.debug.print("size={d}\n", .{@sizeof(i32)}); // 4
std.debug.print("align={d}\n", .{@alignOf(i32)}); // 4
```

2.3 defer and errdefer

```
const std = @import("std");

// defer — runs when the enclosing scope exits (success OR failure)
// Executed in LIFO (last in, first out) order
fn processFile(path: []const u8) !void {
    const file = try std.fs.cwd().openFile(path, .{});
    defer file.close(); // guaranteed to run

    const allocator = std.heap.page_allocator;
    const buf = try allocator.alloc(u8, 4096);
    defer allocator.free(buf); // also guaranteed

    const bytes_read = try file.read(buf);
    std.debug.print("Read {d} bytes\n", .{bytes_read});
    // Both defer statements run here — file.close() then allocator.free()
}

// errdefer — ONLY runs when the function returns an error
// Critical for cleanup in initialisation functions
fn initServer(allocator: Allocator, port: u16) !Server {
    const socket = try createSocket(port);
    errdefer socket.close(); // close socket if anything below fails

    const buf = try allocator.alloc(u8, 64 * 1024);
    errdefer allocator.free(buf); // free buffer if anything below fails

    try socket.bind();
    try socket.listen(128);

    return Server{ .socket = socket, .buf = buf };
    // If we reach here, no errdefer runs — caller owns the resources
}

// defer with a block for complex cleanup
defer {
    std.debug.print("Cleaning up...\n", .{});
    database.disconnect();
    log.flush();
}
```

Phase 2 Projects



Arena-Based String Builder

Build a string builder using `ArenaAllocator`: `append`, `prepend`, `insert`, `replace`, `split`, `join`. All operations allocate from the arena. `deinit()` frees everything at once.

Concepts: `ArenaAllocator` · `slices` · `defer` · `std.mem` · `ArrayList`

PHASE 3 Error Handling*Error sets, error unions, try, catch — explicit by design*

3.1 Error Sets and Error Unions

Zig errors are values, like in Go, but they are more structured. An error set is an enum of named errors. A function that can fail returns an error union: `T` or an error from a set.

```
// Defining an error set
const FileError = error{
    FileNotFound,
    PermissionDenied,
    UnexpectedEOF,
    InvalidFormat,
};

// Error union type: !T (inferred set) or FileError!T (explicit set)
fn openConfig(path: []const u8) FileError![]u8 {
    // ...
    return FileError.FileNotFound;
}

// Inferred error sets — let Zig figure out what errors the function can return
fn readLine(file: std.fs.File) ![]u8 {
    // The ! prefix means: the error set is inferred from the function body
    return file.reader().readUntilDelimiterAlloc(
        std.heap.page_allocator, '\n', 1024
    );
}

// Merging error sets
const AppError = FileError || std.mem.Allocator.Error;

// anyerror — the global error set (superset of all errors)
// Avoid when possible — loses type information
fn riskyFn() anyerror!void { }
```

3.2 try, catch, and Handling Errors

```
// try — propagate error up if it is an error
// try expr is equivalent to: expr catch |err| return err
fn loadConfig(allocator: Allocator, path: []const u8) !Config {
    const file = try std.fs.cwd().openFile(path, .{}); // propagates on error
    defer file.close();
}
```

```

const content = try file.reader().readAllAlloc(allocator, 1_000_000);
errdefer allocator.free(content);

return try parseConfig(content); // propagates parse errors
}

// catch - handle or transform the error
const val = riskyOp() catch |err| switch (err) {
    error.NotFound => defaultValue,
    error.Timeout  => blk: {
        log.warn("timeout, using default");
        break :blk defaultValue;
    },
    else           => return err, // propagate unexpected errors
};

// catch with a default value
const port = parsePort(portStr) catch 8080;

// catch unreachable - assert the error can't happen (panic if wrong)
const x = parseInt("42") catch unreachable;

// Handling errors in main
pub fn main() !void {
    loadConfig(allocator, "config.json") catch |err| {
        std.debug.print("Failed to load config: {}\n", .{err});
        std.process.exit(1);
    };
}

// if on an error union - like Rust's if let
if (riskyOp()) |value| {
    std.debug.print("Success: {d}\n", .{value});
} else |err| {
    std.debug.print("Error: {}\n", .{err});
}

```

Phase 3 Projects



INI Config Parser

Parse INI files: sections, key=value pairs. Return typed errors for parse failures (MalformedKey, MissingSectionHeader, DuplicateKey). Use errdefer for cleanup on failure. Full test suite.

Concepts: error sets · error unions · try · catch · errdefer · testing

PHASE 4 Comptime

Compile-time execution — Zig's most powerful feature

4.1 What is Comptime?

comptime is Zig's way of running code at compile time. It replaces C macros, C++ templates, and runtime reflection. If you can compute something at compile time, Zig lets you do it with the same language — no special template syntax.

```

const std = @import("std");

// comptime parameter — type is computed at compile time
fn makeArray(comptime T: type, comptime len: usize) [len]T {
    return std.mem.zeroes([len]T);
}

const ints    = makeArray(i32, 10); // [10]i32 of zeros
const floats  = makeArray(f64, 5);  // [5]f64 of zeros

// Generic function via comptime type parameter
fn max(comptime T: type, a: T, b: T) T {
    return if (a > b) a else b;
}

const m1 = max(i32, 3, 7); // 7
const m2 = max(f64, 1.5, 2.5); // 2.5

// anytype — infer type at compile time (more ergonomic than comptime T: type)
fn print(value: anytype) void {
    std.debug.print("{any}\n", .{value});
}

print(42); // works
print("hello"); // works
print(3.14); // works

// comptime block — arbitrary code at compile time
const Fibonacci = blk: {
    var arr: [20]u64 = undefined;
    arr[0] = 0; arr[1] = 1;
    var i: usize = 2;
    while (i < 20) : (i += 1) { arr[i] = arr[i-1] + arr[i-2]; }
    break :blk arr;
};

// Fibonacci is a compile-time constant array — no runtime cost at all
std.debug.print("{d}\n", .{Fibonacci[10]}); // 55

```

4.2 Type Introspection with @typeInfo

```

const std = @import("std");

// @TypeOf — get the type of an expression at compile time
const x: i32 = 5;
const T = @TypeOf(x); // i32

// @typeInfo — get detailed information about a type
fn printTypeInfo(comptime T: type) void {
    const info = @typeInfo(T);
    switch (info) {
        .Int => |i| std.debug.print("Int: {d}bits signed={any}\n", .{i.bits,
i.signedness == .signed}),
        .Float => |f| std.debug.print("Float: {d}bits\n", .{f.bits}),
        .Struct => |s| std.debug.print("Struct with {d} fields\n",
.{s.fields.len}),
        .Enum => |e| std.debug.print("Enum with {d} values\n",
.{e.fields.len}),
        .Pointer => std.debug.print("Pointer\n", .{}),
        else => std.debug.print("Other\n", .{}),
    }
}

// Generic serialiser — works for any struct
fn jsonSerialise(comptime T: type, value: T, writer: anytype) !void {
    const info = @typeInfo(T);
    if (info != .Struct) @compileError("jsonSerialise only supports structs");

    try writer.writeByte('{');
    inline for (info.Struct.fields, 0..) |field, i| {
        if (i > 0) try writer.writeAll(", ");
        try writer.print("\"{s}\": ", .{field.name});
        try writer.print("{any}", .{@field(value, field.name)});
    }
    try writer.writeByte('');
}

const Point = struct { x: f32, y: f32 };
var buf: [256]u8 = undefined;
var stream = std.io.fixedBufferStream(&buf);
try jsonSerialise(Point, .{ .x = 1.0, .y = 2.0 }, stream.writer());
// output: {"x": 1.0, "y": 2.0}

```

4.3 Comptime Interfaces — Duck Typing at Compile Time

```

// Zig has no interfaces as a built-in concept.
// Instead, you use comptime to achieve polymorphism.

// Method 1: anytype parameter (simplest)
fn writeAll(writer: anytype, data: []const u8) !void {
    // 'writer' just needs to have a writeAll method
    // If it doesn't, you get a helpful compile error
    try writer.writeAll(data);
}

```

```

}

// Method 2: explicit interface check (self-documenting)
fn Writer(comptime T: type) type {
    // Verify T has a writeAll method with the right signature
    if (!@hasDecl(T, "writeAll")) {
        @compileError("type must have writeAll method");
    }
    return T;
}

// Method 3: tagged union for runtime polymorphism
const ShapeTag = enum { circle, rect };
const Shape = union(ShapeTag) {
    circle: Circle,
    rect: Rect,

    fn area(self: Shape) f64 {
        return switch (self) {
            .circle => |c| std.math.pi * c.radius * c.radius,
            .rect    => |r| r.w * r.h,
        };
    }
};

const shapes = [_]Shape{
    .{ .circle = .{ .radius = 5 } },
    .{ .rect    = .{ .w = 4, .h = 6 } },
};

for (shapes) |s| { std.debug.print("area={d:.2}\n", .{s.area()}); }

```

Phase 4 Projects



Generic HashMap

Build a generic HashMap(K, V) using comptime type parameters. Open addressing with linear probing, load factor resize, comptime verification that K is hashable.

Concepts: comptime · @typeInfo · generics · anytype · @compileError



Compile-Time JSON Serialiser

A serialiser that uses @typeInfo to inspect any struct at compile time and generate JSON. No runtime type checking — all field access is inlined at compile time via inline for.

Concepts: @typeInfo · inline for · @field · anytype · comptime

PHASE 5 Structs, Enums & Unions*Zig's type system — packed, extern, tagged unions*

5.1 Structs

```
// Basic struct
const Point = struct {
    x: f32,
    y: f32,

    // Methods - first parameter is the receiver
    pub fn distance(self: Point, other: Point) f32 {
        const dx = self.x - other.x;
        const dy = self.y - other.y;
        return @sqrt(dx * dx + dy * dy);
    }

    // Associated function (no self)
    pub fn origin() Point {
        return .{ .x = 0, .y = 0 };
    }
};

// Struct initialisation - .{} syntax
const p1 = Point{ .x = 1.0, .y = 2.0 };
const p2 = Point.origin();
std.debug.print("{d:.2}\n", .{p1.distance(p2)});

// Anonymous struct (tuple-like)
const pair = .{ @as(i32, 1), @as(f64, 2.5) };
std.debug.print("{d} {d:.1}\n", .{pair[0], pair[1]});

// packed struct - exact memory layout (for hardware registers, protocols)
const Flags = packed struct {
    read:    bool = false,
    write:   bool = false,
    execute: bool = false,
    _pad:    u5    = 0,    // padding bits
};
// @sizeof(Flags) == 1 (8 bits total)
var f = Flags{ .read = true, .write = true };

// extern struct - C-compatible memory layout
const CPoint = extern struct { x: f32, y: f32 };
// Can be passed directly to C functions
```

5.2 Enums


```
// Simple enum
const Direction = enum { north, south, east, west };
const dir = Direction.north;

// Enum with methods
const Colour = enum {
    red, green, blue,

    pub fn toHex(self: Colour) u24 {
        return switch (self) {
            .red    => 0xFF0000,
            .green  => 0x00FF00,
            .blue   => 0x0000FF,
        };
    }
};

// Enum with explicit integer backing type (for C interop or protocols)
const Status = enum(u8) {
    ok          = 200,
    notFound    = 404,
    error_      = 500,

    pub fn isSuccess(self: Status) bool {
        return @intFromEnum(self) < 400;
    }
};
const s = Status.ok;
std.debug.print("{d}\n", .{@intFromEnum(s)}); // 200
const fromInt = @enumFromInt(Status, 404); // Status.notFound

// Non-exhaustive enum — allows unknown values (for C bindings)
const CEnum = enum(c_int) {
    A = 0, B = 1, C = 2,
    _, // allows unknown values
};
```

5.3 Tagged Unions — The Most Important Zig Type

```
// A tagged union holds one of several types, with a discriminant tag
// This is Zig's primary polymorphism mechanism

const JsonValue = union(enum) {
    null_value: void,
    boolean:    bool,
    integer:    i64,
    float:      f64,
    string:     []const u8,
    array:      []JsonValue, // recursive — needs allocation
};
```

```
fn printJson(value: JsonValue) void {
    switch (value) {
        .null_value => std.debug.print("null", .{}),
        .boolean    => |b| std.debug.print("{} ", .{b}),
        .integer     => |i| std.debug.print("{d}", .{i}),
        .float       => |f| std.debug.print("{d:.3}", .{f}),
        .string      => |s| std.debug.print("\"{s}\"", .{s}),
        .array       => |a| {
            std.debug.print("[", .{});
            for (a, 0..) |item, i| {
                if (i > 0) std.debug.print(", ", .{});
                printJson(item);
            }
            std.debug.print("]", .{});
        },
    }
}

const v1 = JsonValue{ .string = "hello" };
const v2 = JsonValue{ .integer = 42 };
const v3 = JsonValue{ .boolean = true };
printJson(v1); // "hello"
printJson(v2); // 42
```

PHASE 6 Standard Library*std.ArrayList, std.HashMap, std.io, std.fs, testing*

6.1 std.ArrayList — Dynamic Arrays

```

const std = @import("std");
const ArrayList = std.ArrayList;

pub fn main() !void {
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit();
    const allocator = gpa.allocator();

    // ArrayList(T) — dynamic array
    var list = ArrayList(i32).init(allocator);
    defer list.deinit();

    try list.append(1);
    try list.append(2);
    try list.append(3);
    try list.appendSlice(&[_]i32{4, 5, 6});

    _ = list.orderedRemove(1); // remove index 1 (value 2), preserves order
    _ = list.swapRemove(0);    // remove index 0, swaps with last (fast)

    std.debug.print("len={d}\n", .{list.items.len});
    for (list.items) |item| {
        std.debug.print("{d} ", .{item});
    }

    // Convert to owned slice (caller must free)
    const slice = try list.toOwnedSlice();
    defer allocator.free(slice);
}

```

6.2 std.StringHashMap and std.AutoHashMap

```

const std = @import("std");

pub fn main() !void {
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit();
    const allocator = gpa.allocator();

    // StringHashMap — keys are []const u8
    var map = std.StringHashMap(u32).init(allocator);
}

```

```
defer map.deinit();

try map.put("alice", 30);
try map.put("bob", 25);

if (map.get("alice")) |age| {
    std.debug.print("alice is {d}\n", .{age});
}

// getOrPut - insert if not present, return entry either way
const result = try map.getOrPut("carol");
if (!result.found_existing) { result.value_ptr.* = 28; }

// Iterate
var it = map.iterator();
while (it.next()) |entry| {
    std.debug.print("{s}: {d}\n", .{entry.key_ptr.*, entry.value_ptr.*});
}

// AutoHashMap - for non-string keys (uses @hashFn automatically)
var intMap = std.AutoHashMap(i32, []const u8).init(allocator);
defer intMap.deinit();
try intMap.put(1, "one");
try intMap.put(2, "two");
}
```

6.3 Testing in Zig

```
const std = @import("std");

// Functions to test
fn add(a: i32, b: i32) i32 { return a + b; }

fn divide(a: f64, b: f64) !f64 {
    if (b == 0.0) return error.DivisionByZero;
    return a / b;
}

// Tests are defined with 'test' blocks - run with: zig test file.zig
test "add works correctly" {
    try std.testing.expectEqual(@as(i32, 5), add(2, 3));
    try std.testing.expectEqual(@as(i32, -1), add(2, -3));
    try std.testing.expectEqual(@as(i32, 0), add(0, 0));
}

test "divide returns error on zero" {
    const result = divide(10.0, 0.0);
    try std.testing.expectError(error.DivisionByZero, result);
}

test "divide works for normal values" {
    const result = try divide(10.0, 4.0);
    try std.testing.expectApproxEqRel(2.5, result, 0.0001);
}
```

```
}

test "allocator detects leaks" {
    // std.testing.allocator automatically checks for leaks
    const allocator = std.testing.allocator;
    const buf = try allocator.alloc(u8, 64);
    // Forgetting defer allocator.free(buf) would fail this test with leak
    detection
    defer allocator.free(buf);
    try std.testing.expect(buf.len == 64);
}

// zig test src/main.zig
// zig test src/main.zig --test-filter "add"
// zig test src/main.zig -freference-trace (better error messages)
```

Phase 6 Projects



Word Frequency Counter

Count word frequencies in a text file using `StringHashMap`. Sort results by frequency. Handle UTF-8 correctly with `std.unicode`. Test with `std.testing.allocator`.

Concepts: [StringHashMap](#) · [std.fs](#) · [std.io](#) · [ArrayList](#) · [std.testing](#) · [sort](#)

PHASE 7 Build System, C Interop & Ecosystem

build.zig, cross-compilation, and calling C from Zig

7.1 The Zig Build System

```
// build.zig — the build script, written in Zig itself
const std = @import("std");

pub fn build(b: *std.Build) void {
    const target = b.standardTargetOptions(.{});
    const optimize = b.standardOptimizeOption(.{});

    // Build a binary
    const exe = b.addExecutable(.{
        .name = "myapp",
        .root_source_file = b.path("src/main.zig"),
        .target = target,
        .optimize = optimize,
    });
    b.installArtifact(exe);

    // Run: zig build
    // Cross-compile: zig build -Dtarget=aarch64-linux-musl
    // Release: zig build -Doptimize=ReleaseFast

    // Add tests
    const tests = b.addTest(.{
        .root_source_file = b.path("src/main.zig"),
        .target = target, .optimize = optimize,
    });
    const run_tests = b.addRunArtifact(tests);
    const test_step = b.step("test", "Run tests");
    test_step.dependOn(&run_tests.step);

    // Link a C library
    exe.linkSystemLibrary("curl");
    exe.linkLibC();

    // Use zig-fetch for dependencies
    // zig fetch --save https://example.com/lib.tar.gz
}

// Common build commands:
// zig build -- build in debug mode
// zig build -Doptimize=Debug -- explicit debug (safety checks on)
// zig build -Doptimize=ReleaseSafe -- optimised with safety
// zig build -Doptimize=ReleaseFast -- maximally fast, no safety
// zig build -Doptimize=ReleaseSmall -- smallest binary size
// zig build test -- run tests
```

7.2 C Interoperability

```
// Calling C from Zig — zero-cost, no wrapper needed
const c = @cImport({
    @cInclude("stdio.h");
    @cInclude("stdlib.h");
    @cInclude("string.h");
});

pub fn main() void {
    // Call C functions directly
    _ = c.printf("Hello from C printf!\n");

    // C malloc/free via Zig
    const ptr = c.malloc(64) orelse @panic("malloc failed");
    defer c.free(ptr);

    // C strings
    const hello: [*:0]const u8 = "hello";
    const len = c.strlen(hello);
    _ = c.printf("Length: %zu\n", len);
}

// Linking a C library (in build.zig):
// exe.linkLibC();
// exe.linkSystemLibrary("sqlite3");

// Exporting Zig functions for C to call
export fn addNumbers(a: c_int, b: c_int) c_int {
    return a + b;
}

// compile with: zig build-lib mylib.zig -lc
// generates: libmylib.so (or .a) with C-compatible symbols

// zig cc — use Zig as a C compiler (cross-compiles any C code)
// zig cc -o hello hello.c -target aarch64-linux-musl
// This is how you bootstrap Zig into existing C projects
```

7.3 The Zig Ecosystem

Category	Package / Tool	Purpose
Package Registry	pkg.zig.fyi / build.zig.zon	Community packages via zig fetch
HTTP Client	std.http.Client (built-in)	HTTP/1.1 and HTTPS in the standard library
Web Framework	http.zig	Fast HTTP server framework
JSON	std.json (built-in)	Parsing and serialising — part of std
Database	zig-sqlite, pg.zig	SQLite and PostgreSQL bindings

Async Runtime	libxev	Cross-platform async I/O (io_uring on Linux)
CLI	zig-clap	Command line argument parsing
Build Tools	zigmod, zpm	Alternative package managers
WASM	Built-in target	zig build -Dtarget=wasm32-wasi
Embedded	microzig	HAL and drivers for microcontrollers

Capstone Projects



Tiny HTTP Server

A minimal HTTP/1.1 server from scratch using `std.net.TcpServer`: accept connections, parse request line and headers, route GET and POST, serve files from disk. No external libraries.

Concepts: [std.net](#) · [std.http](#) · [std.fs](#) · [error unions](#) · [tagged unions](#) · [comptime](#)



Rewrite a C Tool in Zig

Take a small C utility (e.g. `wc`, `cat`, `head`) and rewrite it in Zig. Then cross-compile it for Linux ARM64, Windows, and macOS from a single machine. Compare binary size and performance.

Concepts: [C interop](#) · [build.zig](#) · [cross-compilation](#) · [std.fs](#) · [std.io](#) · [allocators](#)

You have reached the end of the Zig coursework.

Zig rewards patience. Fight the compiler — it is trying to help you. Then write C interop and feel the power.